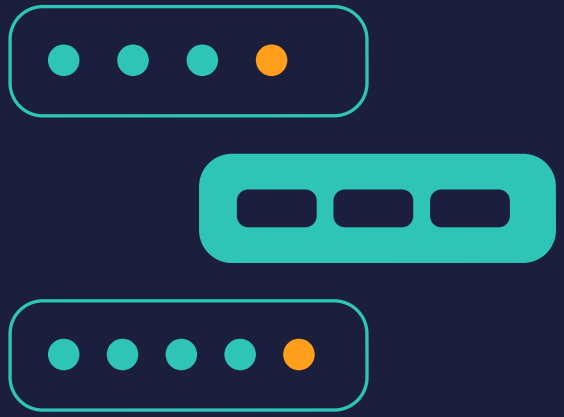
دوره مهندسی LLM و هوش مصنوعی مولد

جلسه ۲

وایب کدینگ: برنامه‌نویسی با دستیار هوش مصنوعی



ابزارها، گردش کار حرفه‌ای، مشخصات و تست، فایل قواعد پروژه و git

کیوان محمودی

در این جلسه

۱ وایب کدینگ چیست؟

۲ طیف ابزارها

۳ گردش کار حرفه‌ای

۴ مشخصات و تست

۵ یک دستیار کدنویس کوچک

۶ بازبینی کد

۷ فایل قواعد پروژه

۸ git به‌عنوان تور ایمنی

وایب کدینگ: دو برداشت

وایب کدینگ خالص

- فقط نتیجه دیده می‌شود، نه کد
- سرعت بسیار بالا
- مناسب نمونه‌سازی و آزمایش
- خطرناک برای محصول

مهندسی با کمک هوش مصنوعی

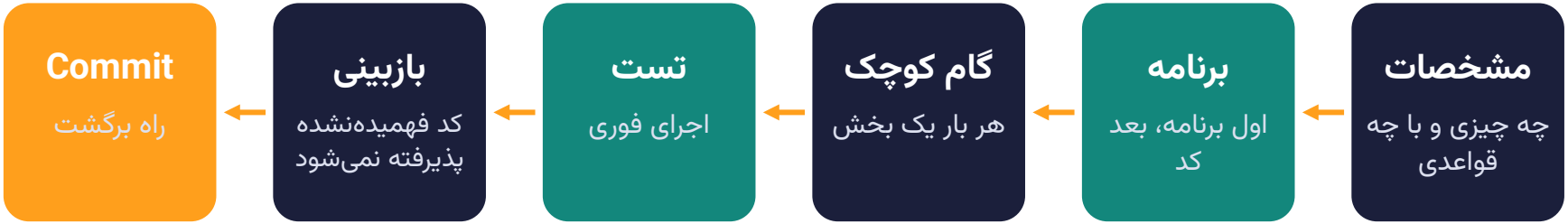
- کد خوانده و فهمیده می‌شود
- تست و بازبینی اجباری است
- مسئولیت با برنامه‌نویس است
- مناسب محصول واقعی

طیف ابزارها

نقش برنامه‌نویس	کار ابزار	نمونه ابزار	سطح
می‌نویسد و می‌پذیرد یا رد می‌کند	پیشنهاد ادامه خط و تابع	Copilot، Continue	**تکمیل خودکار**
می‌پرسد و جای‌گذاری می‌کند	پاسخ و تولید تکه کد	Cursor Chat، Copilot Chat	**چت در ویرایشگر**
هدف می‌دهد و بازبینی می‌کند	تغییر چند فایل، اجرای دستور و تست	Claude Code، Cursor Agent، Codex CLI	**ایجنت**

جایگزین رایگان و همیشه در دسترس: افزونه Continue در VS Code همراه با Ollama

گردش کار حرفه‌ای



- تفاوت حرفه‌ای و تازه‌کار در **اندازه گام‌ها**ست.
- خواستن کل برنامه در یک درخواست یعنی هزار خط کدی که کسی نمی‌فهمد.

مشخصات خوب چه دارد؟

```
parse_price(text: str) -> int

"۱۲٫۵ میلیون تومان"   ->  12_500_000
"۴۵٬۰۰۰ تومان"        ->  45_000
"رایگان"              ->  0
"قیمت ندارد"          ->  ValueError
```

- امضای دقیق تابع یا رابط: ورودی، خروجی و نوع‌ها.
- قواعد رفتاری، به‌خصوص **موارد مرزی**: ارقام فارسی، ورودی خالی، واحدهای مختلف.
- رفتار در خطا: چه زمانی و چه استثنایی.
- محدودیت‌ها: کتابخانه مجاز، نسخه پایتون، سبک کد.

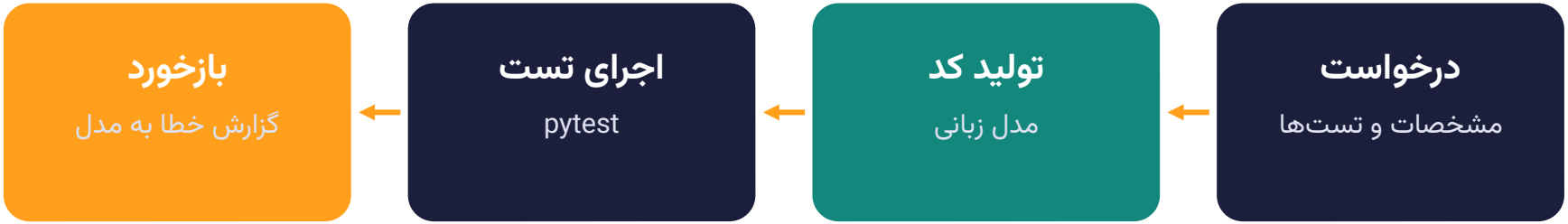
تست‌ها بخشی از مشخصات‌اند

```python
@pytest.mark.parametrize("text, expected", [
    ("45000 تومان", 45_000),
    ("۴۵,۰۰۰ تومان", 45_000),
    ("۲۵۰ هزار تومان", 250_000),
    ("۱۲٫۵ میلیون تومان", 12_500_000),
    ("رایگان", 0),
])
def test_parse_price(text, expected):
    assert parse_price(text) == expected

def test_no_number():
    with pytest.raises(ValueError):
        parse_price("قیمت ندارد")
```

- تست قبل از کد: **TDD**.
- تست مشخصات را بدون ابهام بیان می‌کند.
- تست معیار خودکار پذیرش کد است.
- `parametrize` یک تست را روی چند ورودی اجرا می‌کند.

درون یک ابزار ایجنتی



- این حلقه تا پاس شدن تست‌ها یا رسیدن به سقف تلاش تکرار می‌شود.
- Cursor Agent، Claude Code و Codex همین منطق را با ابزارهای بیشتر اجرا می‌کنند.
- مدل کوچک محلی ممکن است شکست بخورد؛ مدل بزرگ معمولاً در یکی دو تلاش موفق است.

پیاده‌سازی حلقه

```
for attempt in range(1, max_attempts + 1):
    reply = chat(client, MODEL, messages=messages)
    write_file("solution.py", extract_code(reply))
    passed, report = run_tests()
    if passed:
        return True
    messages += [
        {"role": "assistant", "content": reply},
        {"role": "user",
         "content": f"Tests failed:\n{report}\nFix it."},
    ]
return False
```

- پاسخ مدل در تاریخچه می‌ماند تا اصلاح بعدی را ببیند.
- گزارش کامل خطا مهم‌ترین ورودی برای اصلاح است.
- سقف تلاش از حلقه بی‌پایان و هزینه زیاد جلوگیری می‌کند.

بازبینی: تست سبز کافی نیست

- آیا کد را **کامل می‌فهمم**؟ اگر نه، توضیح یا ساده‌سازی بخواهید.
- چه ورودی‌هایی تست نشده‌اند؟ «۱٬۲۵۰٬۰۰۰ تومان»، «دو میلیون تومان».
- آیا ابزار قابلیت اضافه‌ای ساخته که کسی نخواسته؟
- آیا تابع یا کتابخانه‌ای استفاده شده که وجود ندارد؟
- هر یافته جدید به مشخصات و تست‌ها اضافه می‌شود: **مشخصات زنده است**.

فایل قواعد پروژه

- **CLAUDE.md** برای Claude Code، **AGENTS.md** برای Codex و دیگران.
- پوشه `cursor/rules.` برای Cursor.
- در شروع هر جلسه کاری خوانده می‌شود.
- حافظه پروژه: هر چیزی که نمی‌خواهید تکرار کنید.

```
# Project rules

## Commands
- Run tests: `python -m pytest -q`

## Conventions
- Type hints on every public function.
- User-facing text is Persian; code is English.
- Never hard-code API keys.

## Workflow
- Propose a plan before changing many files.
- Write tests for every behaviour change.
- Do not add dependencies without asking.
```

الگوهای مؤثر درخواست

بنویسید	به جای
«در models.py کلاس Product با این فیلدها بساز؛ فعلاً فقط مدل داده.»	**«یک فروشگاه اینترنتی بساز»**
«این تست با این پیام شکست می‌خورد؛ اول علت را توضیح بده.»	**«این باگ را درست کن»**
«تابع search را خواناتر کن؛ رفتار و تست‌ها نباید تغییر کنند.»	**«کد را بهتر کن»**
«برای parse_price تست بنویس؛ ورودی خالی و ارقام فارسی را پوشش بده.»	**«تست بنویس»**

git: تور ایمنی

کاربرد	دستور
دیدن دقیق تغییرات ابزار پیش از پذیرش	**git status / git diff**
پذیرش تکه‌تکه تغییرات	**git add -p**
ثبت هر گام سالم	**git commit**
دور ریختن تغییرات ثبت‌نشده	**git restore .**
کار روی شاخه جدا برای تغییرات بزرگ	**git checkout -b feature**

قاعده طلایی: قبل از هر درخواست بزرگ از ابزار، کار فعلی را commit کنید.

جمع‌بندی

- سه سطح ابزار: تکمیل خودکار، چت و ایجنت؛ اختیار بیشتر یعنی بازبینی مهم‌تر.
- گردش کار: مشخصات، برنامه، گام کوچک، تست، بازبینی و commit.
- تست‌های از پیش نوشته‌شده دقیق‌ترین مشخصات‌اند.
- ابزارهای ایجنتی یک حلقه دارند: تولید، اجرا، خطا، اصلاح.
- فایل قواعد پروژه حافظه پروژه برای ابزار است.

تمرین

۱ دو مورد آزمایشی جدید اضافه کنید و حلقه را دوباره اجرا کنید.

۲ حلقه را طوری تغییر دهید که پیش از کد، برنامه پنج‌خطی بخواهد.

۳ تابع normalize_mobile را با مشخصات و تست پیاده‌سازی کنید.

۴ با Continue و Ollama اسکریپت ارزان‌ترین کالای هر دسته را بنویسید.